Key Recovery Attack on 4-Round Custom DES Using Differential Cryptanalysis

1. Introduction

This report documents a successful differential cryptanalysis attack on a custom 4-round DES variant implemented with weakened S-boxes. The objective was to recover the complete 56-bit secret key using only a small number of plaintext-ciphertext (PT-CT) pairs obtained through oracle queries, by exploiting the cryptanalytic weaknesses introduced by the reduced round count and specially crafted S-boxes.

The 4-round design represents a significant departure from the standard 16-round DES cipher and provides an ideal platform to demonstrate the fundamental principles of differential cryptanalysis as originally presented by Biham and Shamir. By careful analysis of S-box differential properties and strategic plaintext pair selection, I successfully recovered the master key with minimal oracle queries.

---

2. S-Box Analysis and Differential Distribution

2.1 Differential Distribution Table Construction

The foundation of differential cryptanalysis is the Differential Distribution Table (**DDT**), which characterizes the behavior of S-boxes under input differences. For each S-box, I constructed a complete DDT by computing, for every possible 6-bit input XOR difference ΔX (ranging from 0 to 63), which 4-bit output XOR differences ΔY (ranging from 0 to 15) are possible and with what frequency.

Formally, **DDT[ΔX][ΔY]** counts the number of input pairs (x, x') such that

$$\mathbf{x \oplus x' = \Delta X} \text{ and } \mathbf{S(x) \oplus S(x') = \Delta Y}$$

Since there are 64 possible input pairs for each ΔX, the probability of a specific differential $\mathbf{\Delta X \rightarrow \Delta Y}$ is the count divided by 64.

2.2 Key Findings: Weakened S-Box Properties

The analysis revealed that the weakened S-boxes contain significantly higher probability differentials than standard DES S-boxes. Standard DES S-box differentials typically have maximum probabilities around 14-16%, whereas these weakened S-boxes exhibit differentials with probabilities up to 50-53%, particularly in S-boxes 6 and 7. This orders-of-magnitude difference in differential bias is the primary cryptographic weakness we can exploit.

For instance:

- S-Boxes 6 and 7 exhibits several differentials with probabilities up to 50%
- S-Box 7 shows exceptional weakness with a 53.125% probability differential (input 0x01 → output 0x03)
- S-Box 8 has exploitable differentials around 40.6%

This concentration of high-probability differentials in specific S-boxes is not accidental - it directly stems from the design of the weakened S-boxes provided.

---

3. Differential Characteristic Selection

3.1 The Characteristic Design Principle

A differential characteristic specifies a sequence of XOR differences through the cipher that has a certain probability of occurring. For a successful 4-round attack, we require a characteristic where the first three rounds have predictable and analyzable behavior, leaving the fourth round to be attacked directly.

The attack employs characteristics with **probability 1.0 through the first round**, which is achieved by ensuring that the right half of the plaintext XOR is zero. When $R_0 \oplus R_0' = 0$ (meaning the right halves are identical), the Feistel function $F(R,K)$ becomes identical for both plaintexts regardless of the round key: $F(R_0,K) = F(R_0',K)$. This guarantees that the output of round 1 follows the characteristic with complete certainty.

3.2 Plaintext Differential Selection

3 primary plaintext differentials are used in the attack, each designed to activate a single S-box in the second round to get completely unrelated (PT, CT) pairs and for maximum signal clarity:

> **0x0000200000000000 ; 0x0000400000000000 ; 0x0000600000000000**
> — All these activate S-box 5 in round 2

The specific hex values were selected after careful analysis of the expansion permutation (E-table) and S-box input distributions. Each value, when expanded through the expansion permutation and XORed with plaintext differences, produces a non-zero 6-bit input difference to exactly one S-box in round 2.

3.3 Why Single S-Box Activation Matters

By carefully choosing ΔP such that only one S-box is activated in round 2, we minimize noise in our differential trail. If multiple S-boxes were activated, the combination of their independent probabilistic differentials would significantly lower the overall characteristic probability. Furthermore, single S-box activation provides a clean, analyzable signal when we perform the last-round attack: the output difference from round 2 through the final round is influenced primarily by this one high-probability differential, allowing us to identify the correct round 4 subkey bits with high confidence.

Selecting differentials that produce non-zero input to exactly one S-box (matching one of the high-probability differentials identified in our DDT analysis) ensures maximum attack efficiency.

---

4. Key Recovery Methodology

4.1 Last-Round Attack and Subkey K4 Recovery

The attack is a last-round differential attack against round 4. With knowledge of the plaintext-ciphertext pairs and the differential characteristic through rounds 1-3, we can infer information about the round-4 subkey K4.

For each ciphertext pair (C1,C2) we observe the ciphertext XOR ΔC = C1⊕C2.

Since the Feistel structure of round 4 is invertible, we can write:

$$R_4 = R_3 \text{ and } L_4 = L_3 \oplus F(R_3, K_4)$$

By partially decrypting the ciphertext through the inverse of the permutation P (used within the Feistel function), we recover the S-box output difference.

Vote Counting Mechanism: For each of the 8 S-boxes (excluding the one activated by the chosen differential), we perform the following for each ciphertext pair:

1. Extract the 6-bit expanded input to the S-box from the ciphertext
2. Extract the 4-bit output difference from the ciphertext XOR
3. For each of the 64 possible 6-bit subkey values $k \in [0,63]$:
    - Compute S-box outputs for the two inputs $i_1 = e_1 \oplus k$ and $i_2 = e_2 \oplus k$
    - Check if the output XOR matches the observed difference
    - If yes, increment the vote count for subkey value k.
4. After processing all pairs, the subkey value with the highest vote count is selected.
5. Here the highest vote should be equal to the number of plaintext ciphertext pairs taken since the probability of the differential for these S-boxes (i.e., probability of **ΔX→ΔY** is 1 since **ΔX=0 => ΔY=0**

This voting mechanism provides robust statistical filtering: correct subkey bits receive consistent votes from multiple pairs, while incorrect guesses accumulate random, uncorrelated votes. With enough pairs, the correct subkey typically emerges as the clear winner in the vote count.

4.2 Master Key Reconstruction from K4

Once K4 is recovered (42 bits, since one S-box is excluded), we reconstruct the complete master key through key schedule inversion. The process follows these steps:

Step 1: Invert PC2 Permutation
The 48-bit K4 is obtained by applying PC2 to a 56-bit intermediate key. We invert this operation to recover 56 bits (with some positions still unknown due to the excluded S-box). K4 contains 48 bits, while the pre-PC2 key contains 56 bits; applying the inverse of PC2 leaves 8 positions undefined.

Step 2: Reverse Key Schedule Shifts
The key schedule in standard DES involves cumulative left rotations of the left and right halves of the PC1-permuted master key. In the given implementation, for 4 rounds, the cumulative rotation is 6 bits (shifts are [1, 1, 2, 2]). We rotate right by 6 positions to reverse this.

Step 3: Invert PC1 Permutation
After reversing shifts, we apply the inverse of PC1 to map the 56-bit key back to the original 64-bit format (including parity bits). The implementation specifies that parity bits are added as zeros.

Step 4: Brute-Force Remaining Bits
After inversion of the key schedule, we have a 64-bit candidate key with 14 unknown positions. These unknown positions correspond to bits that were not constrained by K4 recovery. So, we brute-force all 2^{14} **= 16384** possibilities and test each candidate key by decrypting the collected ciphertext pairs:

For each candidate key and each collected pair:

- Decrypt the ciphertext using the candidate key
- Compare the decrypted plaintext with the original plaintext
- If all decrypted plaintexts match their originals for all pairs, this is the master key

Only one master key will decrypt all collected pairs correctly.

---

5. Implementation Details

5.1 System Architecture

The implementation is organized into three independent modules.

The **sbox_analysis.py** performs analysis, computing Differential Distribution Tables for all 8 S-boxes to identify high-probability differentials.

The main attack engine resides in **recover_key.py**, which orchestrates the entire differential cryptanalysis attack.

The **verify_recovered_key.py** module handles post-attack verification.

5.2 Main Components in Key Recovery

get_plaintext_pairs(count, differential)

Generates the specified number of plaintext pairs satisfying the chosen differential. For each pair, it creates a random plaintext P_1 and computes $P_2 = P_1 \oplus$ differential, ensuring all pairs have the exact XOR difference required by the attack.

generate_pt_ct_pairs(count, differential, pairs_file)

Queries the oracle for each plaintext pair, obtaining the corresponding ciphertexts. Returns and stores all collected (plaintext, ciphertext) pairs to a file for later use in voting and verification.

Vote Counting Implementation

Implemented the vote counting mechanism for K4 recovery. For each of the 7 active S-boxes, it maintains a vote count for each possible 6-bit subkey value (0-63). It iterates through all collected ciphertext pairs, extracts S-box input and output differences from the ciphertext, and increments votes for subkey values whose S-box outputs match the observed differences. The subkey with the maximum vote count(and that is equal to the number of pairs used) is selected as the recovered K4 bits for that S-box.

design_masterkey(k4_bits)

Reconstructs the 56-bit master key from the recovered K4 subkey. It inverts the PC2 permutation to map from 48-bit K4 to 56 bits, reverses the cumulative key schedule shifts (rotating right by 6 bits for 4 rounds), and inverts the PC1 permutation to recover the original key structure. This function produces a 64-bit candidate key with approximately 14 unknown bit positions.

BruteForce and Validate possible value

Performs brute-force search over all 2^14 possible values for the unknown bits. For each candidate master key, it decrypts all collected ciphertexts and compares them with the original plaintexts. The function returns the key that correctly decrypts all pairs. This step provides definitive verification of the recovered key.

---

6. Results

The attack successfully recovered the 56-bit master key **64F55CD5C9B7D7** using less than or equal to 10 plaintext-ciphertext pairs through oracle queries. The recovered key was verified against all collected pairs—each pair, when encrypted using the recovered key, produced the observed ciphertexts.

The attack demonstrates the vulnerability of 4-round DES with weakened S-boxes to differential cryptanalysis. The dramatic reduction in rounds eliminates security-critical avalanche mixing, while the weakened S-boxes introduce exploitable biases. Together, these factors reduce the complexity of breaking the cipher from the original DES security (2^56 exhaustive search) to a polynomial-time attack requiring fewer than 16 oracle queries.

7. Conclusion

This work successfully applied differential cryptanalysis to a 4-round DES variant, recovering the full 56-bit key with minimal queries by exploiting high-probability S-box differentials and careful characteristic selection. The attack validates the theoretical foundations of differential cryptanalysis and demonstrates why design choices, particularly round count and S-box selection are critical to cipher security. The weakened S-boxes and reduced rounds render this cipher completely broken against a well-chosen differential attack.

---

8. References

1. Biham, E., & Shamir, A. (1990). "Differential Cryptanalysis of DES-like Cryptosystems." In *Advances in Cryptology — EUROCRYPT '90*, Lecture Notes in Computer Science, volume 473.
2. National Institute of Standards and Technology. "Data Encryption Standard (DES)." FIPS Publication 46-3.

Report By: GADDAM HRUSHIKESH RAJ SAI

Date: November 25, 2025